Assignment 2: Software Design Specification

System: Casino Blackjack Cheater Detection System

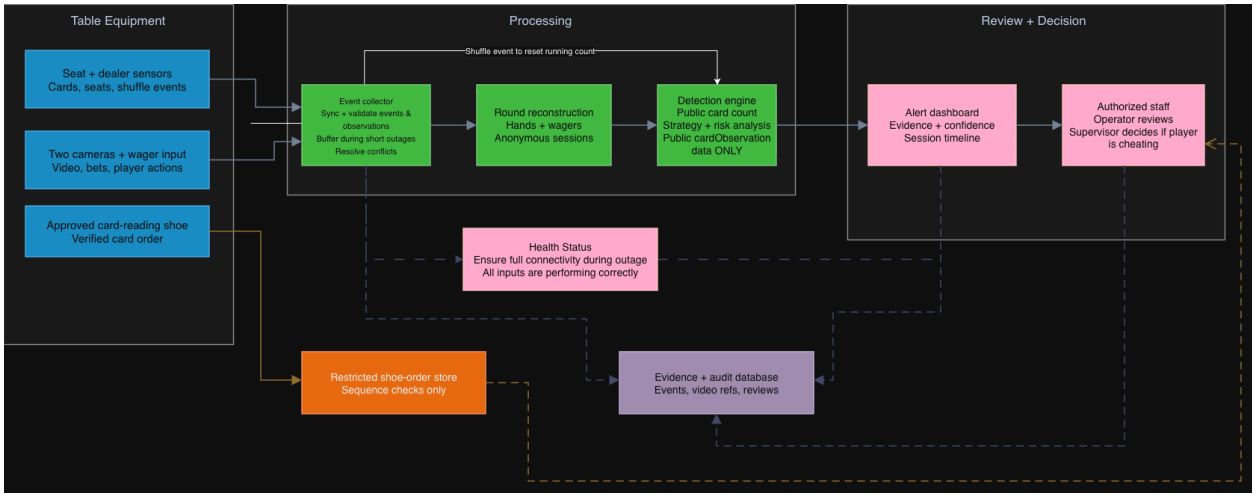
Team Members: Nathan Cubias, Dane Yaddgo, Maksim Makar, Austin Thomas

I. System Description

1. A casino wants a system that helps its surveillance team spot possible card counting and other behavior that may need investigation. Card counters keep track of the high- and low-value cards, or favorable cards, already revealed. They use that information to judge whether the remaining cards favor the player and may adjust their bets or decisions. Since a winning streak or one smart move is not enough evidence to determine whether someone is counting cards, our system will collect and analyze data from equipped blackjack tables and synchronized cameras. It will rebuild each round and look for patterns over time, while staff reviews any concerning activity. Card counting alone is not proof of fraud or illegal cheating, so the system will show the evidence and how confident it is. Casino staff will decide whether any action is allowed under casino policy and applicable law
    - *Sourced from Casino Blackjack Cheater Detection System PDF*

II. Software Architecture Overview

1. Architectural Diagram

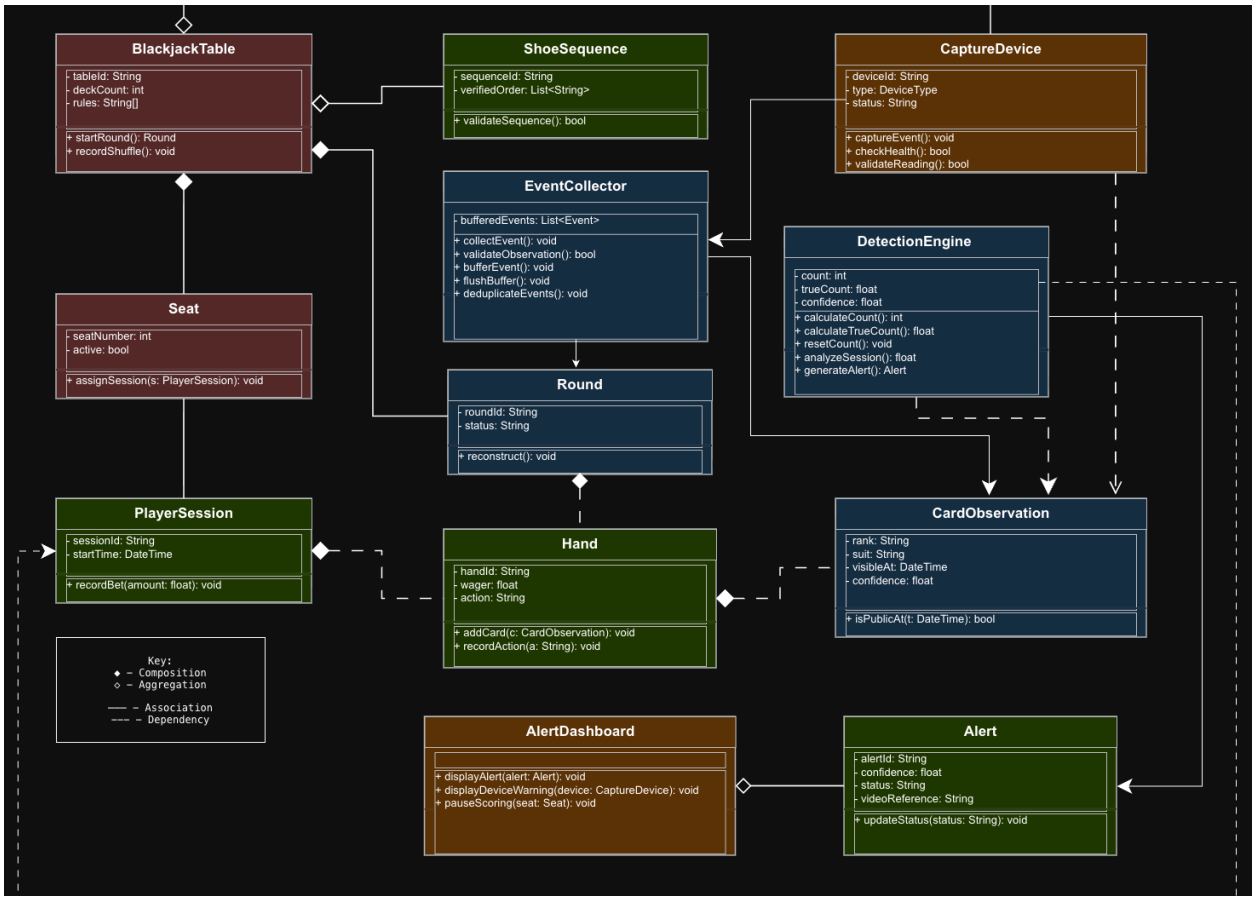


- Explanation:
    - The diagram shows how the system collects and processes information to identify possible card counting. Sensors and cameras collect game data, which is then organized by the event collector and round reconstruction modules. The detection engine analyzes player behavior and generates alerts for casino staff to review. An evidence database stores relevant information and review records, while a separate restricted database stores the verified card order for additional investigations. This foundation allows the system to efficiently build

statistically-correct conclusions.

- Table equipment:
    - The primary data input, designed to gather as much information as possible for the other systems.
- Processing:
    - Analyzes the data from the table equipment to create a running card count and list of current players. This data is referenced against past behavior from the player to identify patterns of cheating.
- Review and Decision:
    - The processed data is stored for employee review with cases with stronger evidence are sent to a pit boss for immediate review.

2. UML Diagram



- Explanation:
    - The UML class diagram details the main classes of the system and how they interact with each other. The BlackjackTable class connects to the table's equipment and contains its seats and rounds. Each seat can have a player session, containing the player's hands. Each hand stores its wager, actions and observed cards. The CaptureDevice class records card observations. Overall, the diagram

utilizes composition, aggregation, association and dependency relationships to display how these classes are connected.

- BlackjackTable
  - Attributes
    - tableID: String
      - Unique identifier for a blackjack table
    - deckCount: int
      - Number of decks used at the table
    - rules: String[]
      - Blackjack rules configured for the table (array)
  - Functions
    - startRound(): Round
      - Creates/returns a new Round object
    - recordShuffle(): void
      - Records shuffle event so system can reset card count
- Seat
  - Attributes
    - seatNumber : Int
      - The identifying number of a seat at a table.
    - active : Boolean
      - Is the seat being used?
  - Functions
    - AssignSession (PlayerSession) : void
      - Assigns a player to the seat.
- Round
  - Attributes
    - roundID : String
      - The rounds unique ID for review purposes
    - Status : String
      - The current status of the round (In Progress, ended, under review)
  - Functions
    - reconstruct() : void
      - Resets the round and the current card count.
- Hand
  - Attributes
    - handID : String
      - The hands unique ID for review purposes
    - Wager : Int
      - The amount wagers on this hand
    - Action : String

            - Any unique actions taken during this hand
    - Functions
        - addCard(CardObservation) : void
            - Deals a card to the hand.
        - recordAction(String) : void
            - Records the current action to the database and sets the new action.
- ShoeSequence
    - Attributes
        - sequenceID: String
            - Unique identifier for card sequence
        - verifiedOrder: List<String>
            - The order of cards in the Shoe
    - Functions
        - validateSequence() : bool
            - Tests the verifiedOrder to see if it is a valid deck

- CaptureDevice:
    - Attributes
        - deviceId: String
            - Unique identifier for device
        - type: DeviceType
            - Identifies the type of device (Camera, card sensor, etc.)
        - status: String
            - Current status
    - Functions
        - captureEvent(): void
            - Captures game information from the device
        - checkHealth(): bool
            - Checks whether the device is functioning properly
        - validateReading(): bool
            - Tests the capture device to see if it is reading correctly

- PlayerSession
    - Attributes
        - sessionId: String
            - Unique identifier for player session
        - startTime: DateTime
            - Date and time when the session began
    - Functions
        - recordBet(amount: float): void

                - Records the players wager using the amount param
- Alert
    - Attributes
        - alertID: String
            - Unique identifier for the alert
        - confidence: float
            - Confidence level of the player cheating
        - status: String
            - The current status of the report, ie. resolved
        - videoReference: String
            - The name of the video the cheating was detected in
    - Functions
        - updateStatus(status: String): void
            - Updates the status of the alert to a new value
- AlertDashboard
    - Attributes
    - Functions
        - displayAlert(alert: Alert): void
            - Prints the alert information to the current console
        - displayDeviceWarning(device: CaptureDevice): void
            - Prints the device failure status to the current console
        - pauseScoring(seat: Seat): void
            - Deactivates the seat to pause monitoring
- CardObservation
    - Attributes
        - rank: String
            - Card rank
        - suit: String
            - Card suit
        - visibleAt: DateTime
            - Date and time when card became publicly visible
        - confidence: float
            - Confidence level of observation (0.0 to 1.0)
    - Functions
        - IsPublicAt(DateTime) : boolean
            - Checks if the card was publicly visible at the specified time.
- DetectionEngine
    - Attributes
        - Count: int
            - Current public card count

        - trueCount: float
            - Current count count, public and private
        - confidence: float
            - Confidence level of observation (0.0 to 1.0)
    - Functions
        - calculateCount() : int
            - Calculates the public count
        - calculateTrueCount() : float
            - Calculates the true count
        - resetCount() : void
            - Resets both counts to 0
        - analyzeSession() : float
            - Develops a confidence level using the current counts
        - generateAlert() : Alert
            - Generates an Alert using the current information
- EventCollector
    - Attributes
        - bufferedEvents: List<Event>
            - A list of unsent events
    - Functions
        - collectEvent() : void
            - Sends a buffered event to the appropriate location
        - validateObservation() : bool
            - Checks if the bufferedEvents include an invalid option
        - bufferEvent() : void
            - Checks if an event has been sent and if not adds it to bufferedEvents
        - flushBuffer() : float
            - Removes all events from the buffer
        - deduplicateEvents() : Alert
            - Goes through the bufferedEvents and removes duplicate options

III. Development Plan And Timeline

Dane Yaddgo: I helped contribute to the process of designing this system. My role was organizing different elements for interaction flow with each other in this design. I helped create the UML Class Diagram, where I listed the classes, their features, and responsibilities in this process. I played an important role in helping with the System Flow Diagram, where I helped with the movement of information from input devices up until processing and then making decisions.

Nathan Cubias: I created the UML diagram and its explanations/descriptions. I also helped create a new PDF file to describe our custom system in full detail.

Austin Thomas: I helped create the Software Architecture Diagram and the descriptions for both the UML and the Software Architecture diagrams in this document.

Maksim: I assisted in making the UML diagram, profiling each class and making its variables and functions. The PDF used to describe the system was also partially created by me.

Task List:

- [x] ~~Complete System PDF~~
- [x] ~~Complete system description~~
- [x] ~~Complete Architecture Diagram~~
- [x] ~~Complete UML Diagram~~
- [x] ~~Write explanations~~
- [x] ~~Write descriptions of each class, attribute, and function~~
- [ ]